

INFORME CONFIDENCIAL

Auditoría de Seguridad Web

Achachaw Books — achachaw.vercel.app

Stack	Next.js 15 · Supabase · Vercel
Fecha	Agosto 2026
Versión	v1.0 — Revisión inicial
Revisado por	Auditoría interna autorizada
Alcance	RLS · PDFs · Admin · Auth · Secretos · Inyección · Lógica de negocio

1	2	3	3
CRÍTICO	ALTO	MEDIO	BAJO

Tabla de contenidos

● H-01	Manipulación de estado de pedido vía RLS UPDATE sin restricción	CRÍTICO
● H-02	Storage RLS de PDFs otorga acceso cruzado entre libros	ALTO
● H-03	proxy.ts es código muerto — no existe middleware.ts	ALTO
● H-04	Monto del pago enviado por el cliente, sin re-validación	MEDIO
● H-05	Tabla category sin RLS habilitado	MEDIO
● H-06	Trigger link_anonymous_orders permite orden-spam sobre cualquier email	MEDIO
● H-07	CSP con unsafe-inline y unsafe-eval — protección XSS neutralizada	BAJO
● H-08	Host header injection en reset de contraseña	BAJO
● H-09	Ruta /api/download/[token] referenciada pero no implementada	BAJO
● Apéndice A	Controles verificados y bien protegidos	APROBADO



Resumen ejecutivo

Estado general del sistema y prioridades de corrección

Se auditó el código fuente completo de **Achachaw Books** (Next.js 15 + Supabase + Vercel), cubriendo migraciones SQL, Server Actions, API Routes, middleware, autenticación, control de acceso a PDFs y lógica de pagos. Se encontraron **9 hallazgos reales** (no teóricos), con severidades desde CRÍTICO hasta BAJO. El hallazgo crítico permite obtener PDFs de pago sin pagar nada.

ID	Hallazgo	Área	Severidad	Acción
H-01	UPDATE de order.status sin restricción de campo	RLS / Lógica	CRÍTICO	Inmediata
H-02	Storage RLS de PDFs permite acceso cruzado	Storage RLS	ALTO	Esta semana
H-03	proxy.ts nunca se ejecuta (no hay middleware.ts)	Middleware	ALTO	Esta semana
H-04	Monto del pago no re-validado contra la BD	Lógica de negocio	MEDIO	Próximo sprint
H-05	Tabla category sin RLS	RLS	MEDIO	Próximo sprint
H-06	Trigger permite spam de órdenes por email	Lógica / Trigger	MEDIO	Próximo sprint
H-07	CSP con unsafe-inline/eval	Headers HTTP	BAJO	Backlog
H-08	Host header injection en password reset	Autenticación	BAJO	Backlog
H-09	Ruta /api/download/[token] sin implementar	API Routes	BAJO	Backlog



Hallazgos de seguridad

Descripción técnica, escenario de explotación y corrección recomendada

1

Manipulación de estado de pedido vía RLS UPDATE sin restricción de campos

CRÍTICO

supabase/migration_05.sql · líneas 37-56 |
src/app/api/leer/[orderId]/[itemId]/route.ts · línea 43

DESCRIPCIÓN

La migración 05 añade la siguiente política RLS sobre la tabla `order`:

```
CREATE POLICY "order: lector actualiza sus pedidos"
ON public."order" FOR UPDATE
USING (auth.uid() = buyer_id)
WITH CHECK (auth.uid() = buyer_id);
```

Esta política **no restringe qué columnas pueden modificarse**. Permite actualizar cualquier campo, incluyendo `status` y `total`, usando el cliente Supabase con la ANON KEY (que es pública por definición al llevar el prefijo `NEXT_PUBLIC_`).

Por otro lado, el endpoint `GET /api/leer/[orderId]/[itemId]` solo verifica:

```
if (order.status !== 'completed' && order.status !== 'processing') {
  return new NextResponse('Payment not verified', { status: 402 })
}
```

No comprueba `order_item.delivery_status`. La misma omisión existe en

`src/app/(main)/leer/[orderId]/[itemId]/page.tsx:31`.

ESCENARIO DE EXPLOTACIÓN (PASOS EXACTOS)

1. El atacante compra cualquier libro → se crea un pedido con `status = 'pending'` y `buyer_id = auth.uid()`.

2. Sin pagar ni subir ningún comprobante, ejecuta desde la consola del navegador (con sesión activa):

```
const sb = createBrowserClient(SUPABASE_URL, ANON_KEY)
await sb.from('order')
  .update({ status: 'completed' })
  .eq('id', '<su-order-id>')
```

3. La política RLS permite el UPDATE porque `auth.uid() = buyer_id`.

4. Visita `/leer/<orderId>/<itemId>` o llama directamente a `/api/leer/<orderId>/<itemId>` → obtiene el PDF completo sin haber pagado nada.

Corrección recomendada:

(a) Eliminar la política UPDATE de `order` para usuarios normales — todas las actualizaciones de pedidos ya ocurren vía `createAdminClient()` en los Server Actions.

(b) Mientras tanto, añadir verificación de `order_item.delivery_status = 'delivered'` en `route.ts:43` y en `leer/page.tsx:31` como defensa en profundidad.

2

Storage RLS de PDFs otorga acceso cruzado entre libros

supabase/migration_02.sql · líneas 43-50

ALTO

DESCRIPCIÓN

La política de lectura sobre el bucket `pdfs` (privado) está definida así:

```
CREATE POLICY "pdfs: comprador con token"
ON storage.objects FOR SELECT USING (
  bucket_id = 'pdfs' AND EXISTS (
    SELECT 1 FROM public.order_item oi
    JOIN public."order" o ON o.id = oi.order_id
    WHERE o.buyer_id = auth.uid()
    AND oi.delivery_status = 'delivered' -- cualquier item entregado
  )
);
```

La condición verifica que el usuario tenga *algún* pedido entregado, pero **no vincula el archivo específico con el pedido**. Cualquier usuario con un libro entregado puede descargar cualquier otro PDF del bucket si conoce la ruta.

La ruta del PDF (`pdf_url`) es visible para todos los usuarios porque la política `"book: lectores ven publicados"` en `migration_01.sql` devuelve toda la fila sin restricción de columnas.

ESCENARIO DE EXPLOTACIÓN

1. Usuario A compra el Libro X (S/ 18) → pedido marcado como `delivered` .
2. Consulta: `await supabase.from('book').select('pdf_url').eq('id', '<id-libro-y>')` → obtiene la ruta en Storage.
3. `await supabase.storage.from('pdfs').download('ruta/del/libro-y.pdf')` → descarga el Libro Y (S/ 25) sin haberlo comprado.

Corrección recomendada:

La política debe correlacionar el path del objeto con el `book.pdf_url` de la compra del usuario. Como alternativa más simple y robusta: mantener el bucket sin políticas SELECT directas y servir todos los PDFs exclusivamente a través de `/api/leer` (que sí valida correctamente). Opcionalmente ocultar `pdf_url` de las políticas SELECT del libro público.

3

proxy.ts nunca se ejecuta — no existe middleware.ts

src/proxy.ts (archivo huérfano)

ALTO

DESCRIPCIÓN

En Next.js, el middleware Edge se activa únicamente si existe el archivo `middleware.ts` (o `src/middleware.ts`). El proyecto tiene `src/proxy.ts` que exporta una función `proxy()` y un `config`, pero **ese archivo nunca es importado ni ejecutado**. No hay ningún `middleware.ts` en el proyecto.

```
// src/proxy.ts - nunca se invoca
export async function proxy(request: NextRequest) { ... }
export const config = { matcher: [...] } // ignorado
```

Consecuencia: no existe ninguna capa de protección centralizada de rutas en el edge. Toda la protección recae sobre chequeos individuales en cada page component y Server Action. Si cualquier página bajo `/dashboard/*` o `/pedidos/*` omite su verificación, queda expuesta sin red de seguridad adicional.

Además, sin middleware no hay logging centralizado de intentos de acceso no autorizado, ni la posibilidad de añadir rate limiting en el edge de forma uniforme.

Corrección recomendada:

Crear `src/middleware.ts` que importe y re-exporte `proxy` y `config` desde `proxy.ts`:

```
export { proxy as default, config } from './proxy'
```

4

Monto del pago enviado por el cliente sin re-validación contra la BD

MEDIO

src/app/(main)/actions.ts · línea 113 |
src/app/(main)/checkout/[orderId]/CheckoutForm.tsx · línea 198

DESCRIPCIÓN

El campo `amount` del registro de pago se construye directamente desde el formulario del cliente:

```
// CheckoutForm.tsx:198
<input type="hidden" name="amount" value={total} />

// actions.ts:113
await admin.from('payment').insert({
  amount: parseFloat(amount), // ← valor enviado por el cliente
})
```

El Server Action no consulta `order.total` desde la base de datos para validar que el monto recibido sea correcto.

ESCENARIO DE EXPLOTACIÓN

1. Usuario inicia checkout de un libro de S/ 25.00.
2. Abre DevTools → Elements → modifica `value="25"` a `value="0.01"` en el campo hidden.
3. Sube un comprobante de Yape de S/ 0.01 (real o falso). La tabla `payment` registra `amount = 0.01`.
Un admin desatento podría aprobar sin comparar contra `order.total`.

Corrección recomendada:

En `submitPayment`, leer el total desde la BD: `const { data: ord } = await admin.from('order').select('total').eq('id', orderId).single()` y usar `ord.total` en el INSERT de payment, ignorando el valor enviado por el cliente.

5

Tabla category sin Row Level Security habilitado

supabase/migration_01.sql · líneas 163-171 (ausencia de la línea para category)

MEDIO

DESCRIPCIÓN

La sección de RLS en migration_01.sql habilita la protección en todas las tablas críticas (`user` , `book` , `order` , `order_item` , `payment` , `comment` , `conversation` , `message`), pero **omite la tabla** `category` . Cuando RLS está deshabilitado, Supabase permite todas las operaciones a todos los roles, incluyendo usuarios anónimos.

```
-- Con RLS deshabilitado, cualquier usuario anónimo puede:  
await supabase.from('category').insert({ name: 'Spam', slug: 'malicious' })  
await supabase.from('category').delete().eq('id', '<uuid-real>')  
await supabase.from('category').update({ name: 'Hacked' }).eq('id', '<uuid-real>')
```

Corrección recomendada:

```
ALTER TABLE public.category ENABLE ROW LEVEL SECURITY;
```

Luego añadir política SELECT pública y restringir INSERT/UPDATE/DELETE solo a admins.

6

Trigger link_anonymous_orders permite asociar órdenes spam a cualquier email

MEDIO

supabase/migration_03.sql · líneas 30-43

DESCRIPCIÓN

El trigger `on_auth_user_registered` vincula pedidos anónimos al nuevo usuario que se registra con el mismo email:

```
UPDATE public."order"  
SET buyer_id = NEW.id  
WHERE buyer_email = NEW.email  
AND buyer_id IS NULL;
```

No hay límite de cuántos pedidos se pueden crear con un email dado, ni validación previa de ese email. Cualquier visitante puede crear pedidos anónimos usando el email de otra persona.

ESCENARIO DE EXPLOTACIÓN

1. Atacante crea 20 pedidos anónimos especificando `buyer_email = 'victima@gmail.com'`.
2. Cuando la víctima se registra con ese correo, el trigger los vincula automáticamente a su cuenta.
3. La víctima ve 20 pedidos desconocidos en su historial. Impacto: confusión, posible bloqueo de cuenta si hay límites por usuario, y ruido en el panel de admin.

Corrección recomendada:

Implementar rate limiting por IP en la creación de pedidos anónimos, y/o requerir verificación del email (OTP de un solo uso) antes de crear un pedido anónimo con ese correo. Como mínimo, limitar a 3-5 pedidos pendientes por email/IP.

7

CSP con unsafe-inline y unsafe-eval neutraliza la protección contra XSS

BAJO

next.config.ts · línea 14

DESCRIPCIÓN

El Content Security Policy configurado en `next.config.ts` incluye:

```
"script-src 'self' 'unsafe-inline' 'unsafe-eval'"
```

La presencia de `'unsafe-inline'` permite ejecución de cualquier script inline (incluyendo `<script>alert(1)</script>`). La presencia de `'unsafe-eval'` permite `eval()` y construcción dinámica de código. Esto invalida completamente la CSP como defensa contra XSS. Los demás headers de seguridad (`X-Frame-Options`, `X-Content-Type-Options`) están correctamente configurados.

Corrección recomendada:

Usar un nonce por request para scripts inline de Next.js. Next.js 14+ soporta CSP con nonce nativamente. Esto elimina la necesidad de `'unsafe-inline'`. `'unsafe-eval'` solo es necesario en desarrollo; en producción puede eliminarse.

8

Host header injection en la URL de reset de contraseña

BAJO

src/app/(auth)/actions.ts · líneas 66-72

DESCRIPCIÓN

El `redirectTo` del email de reset se construye con headers de la solicitud:

```
const host = headersList.get('host') ?? 'localhost:3000'
const proto = headersList.get('x-forwarded-proto') ?? 'http'
const origin = `${proto}://${host}`

await supabase.auth.resetPasswordForEmail(email, {
  redirectTo: `${origin}/api/auth/callback?next=/reset-password`,
})
```

En Vercel, los headers `host` y `x-forwarded-proto` son establecidos por la infraestructura y no son spoofables externamente. Sin embargo, si el proyecto se mueve a un servidor propio o entorno de staging sin proxy confiable, un atacante puede enviar una solicitud con `Host: evil.com` y el email de reset incluirá un link apuntando a `evil.com`.

Corrección recomendada:

Usar una variable de entorno fija: `const origin = process.env.NEXT_PUBLIC_SITE_URL`, eliminando la dependencia de headers controlables por el cliente.

9

Ruta /api/download/[token] referenciada en UI pero no implementada

src/app/(main)/pedidos/[orderId]/page.tsx · líneas 87-92

BAJO

DESCRIPCIÓN

El componente de detalle de pedido genera un link de descarga basado en `download_token`:

```
{item.download_token && (  
  <a href={`\api/download/${item.download_token}`}>  
    Descargar PDF  
  </a>  
)}
```

No existe el archivo `src/app/api/download/[token]/route.ts`. Actualmente el link genera un 404. El riesgo principal es futuro: si esta ruta se implementa sin los controles adecuados (validación de expiración del token, verificación de ownership, uso único), podría convertirse en una vía de acceso no autenticado a PDFs mediante tokens predecibles o reutilizables.

Corrección recomendada:

- (a) Si la funcionalidad no se implementará: eliminar el bloque de código y el campo `download_token` de la consulta.
- (b) Si se implementará: el token debe ser criptográficamente aleatorio, de un solo uso, con expiración corta (ej. 15 minutos), y vinculado explícitamente al `order_item.id` y `user.id`.



Apéndice A — Controles verificados y bien protegidos

Áreas auditadas donde no se encontraron vulnerabilidades explotables

- ✓ **API Route /api/leer — protección de PDFs vía proxy**
Valida UUID, autenticación JWT, ownership de la orden, que el ítem pertenece a la orden, status de la orden, y descarga con admin client. Headers `private, no-store + nosniff`.

- ✓ **Server Actions de administrador**
`requireAdmin()` verifica JWT + rol en BD en cada llamada antes de cualquier operación. No hay protección solo en el frontend.

- ✓ **Upload de comprobantes de pago**
Valida MIME type real (no solo extensión), whitelist de extensiones, límite de 5 MB, UUID del orderId para prevenir path traversal, y verifica ownership del pedido antes de subir.

- ✓ **Prevención de open redirect en login**
El parámetro `next` se valida: debe comenzar con `/`, no con `//` ni contener `://`. Bloquea correctamente redirecciones a dominios externos.

- ✓ **Función search_books_unaccent — sin inyección SQL**
Usa bind parameters en `plpgsql ( LIKE term )`, `SET search_path = ''`, y `SECURITY INVOKER`. RLS del llamador se aplica correctamente.

- ✓ **Secretos no versionados en git**
`.gitignore` excluye `.env*`. `SUPABASE_SERVICE_ROLE_KEY` solo referenciada como env var, nunca hardcodeada. No hay `.env` commiteado.

- ✓ **Service Role Key solo server-side**
`createAdminClient()` en `lib/supabase/admin.ts` solo se importa en Server Components y API Routes. Nunca llega al bundle del cliente.

- ✓ **Aislamiento de pedidos entre usuarios**
La política `"order: lector ve sus pedidos"` usa `auth.uid() = buyer_id`. Un usuario no puede leer ni acceder a pedidos de otro usuario.

- ✓ **Headers HTTP de seguridad**
`X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy: frame-ancestors 'none'`.

- ✓ **RLS habilitado en todas las tablas críticas**
`user`, `book`, `order`, `order_item`, `payment`, `comment`, `conversation`, `message` — todas con RLS activo desde `migration_01`.

- ✓ **Protección de rutas de admin en servidor**
Las páginas admin verifican `profile?.role?.includes('admin')` en el servidor y redirigen si falla. No dependen solo del cliente para ocultar el panel.

- ✓ **Actualización de libros restringida al autor**
En `updateBook`, el UPDATE incluye `.eq('author_id', user.id)` como condición adicional, impidiendo que un escritor modifique libros de otro.

Nota sobre el alcance: Esta auditoría cubre el código fuente disponible en el repositorio. No incluye pruebas de penetración activas contra el entorno de producción, análisis de configuración del proyecto Supabase en el dashboard

(RLS activo real vs. migraciones), ni análisis de dependencias de terceros (npm audit). Se recomienda complementar con una revisión de las políticas efectivas en el Supabase Dashboard > Authentication > Policies.